

7 Algorithm Design & Problem Solving

Cambridge IGCSE Computer Science (2210) – revision notes

In this chapter: program development life cycle · decomposition & abstraction · structure diagrams, flowcharts & pseudocode · standard algorithms (search/sort/totalling/counting) · validation & verification · test data · trace tables · finding & fixing errors.

7.1 The Program Development Life Cycle

Five stages: Analysis → Design → Coding → Testing → Maintenance (Ch7 covers the first four in detail).

Analysis: investigating the problem to produce a requirements specification. Uses two tools:

- Abstraction — keep the key elements needed for the solution, discard unnecessary detail (e.g. a road map only shows what's needed for travel).
- Decomposition — break a complex problem into smaller, more manageable sub-problems (stepwise refinement).

Design: shows HOW the program will work — documented using structure charts, flowcharts, pseudocode.

Coding & iterative testing: writing the program module by module; each module tested, amended, re-tested until it works.

Testing: the whole program run many times with different test data to check every task works together.

Computer systems & sub-systems

A computer system = software + data + hardware + communications + people. Any system can be decomposed into sub-systems (top-down design) until each sub-system does just one action – stepwise refinement.

Every problem/system can be decomposed into 4 component parts:

- Inputs – data entered while the system is active
- Processes – tasks performed on the input (and stored) data
- Outputs – information displayed/printed for the user
- Storage – data kept in files for future use

Example – Alarm app: Inputs: set/cancel alarm, snooze; Processes: compare current time to alarm time, manage snooze; Outputs: alarm sound; Storage: alarm time(s).

7.2 Design Tools

Structure diagrams

Hierarchical diagram showing a system broken into sub-systems, each level giving more detail (top-down design).

Flowcharts

Show the steps (algorithm) and order needed to complete a task. Standard symbols:

Symbol	Meaning
Rounded oval	Terminator – START / STOP
Rectangle	Process – assignment/action, e.g. $A \leftarrow 0$
Parallelogram	Input / Output – e.g. INPUT X / OUTPUT "Error"

Diamond	Decision — always has TWO labelled exits (yes/no)
Arrow	Flow line — direction of flow

Common mistake: forgetting to label BOTH branches of a decision diamond (yes/no) — always label both!

Pseudocode conventions (this syllabus)

- Keywords in CAPITALS (INPUT, IF, FOR...)
- Identifiers start with a capital letter
- Assignment uses $\leftarrow$; indentation of 2 spaces shows nested/repeated statements

```
Cost  $\leftarrow$  10           // assigns 10 to Cost
Price  $\leftarrow$  Cost * 2   // Price = 20
Gender  $\leftarrow$  "M"
Chosen  $\leftarrow$  FALSE
```

IF...THEN...ELSE...ENDIF

```
IF Age < 18
  THEN
    OUTPUT "Child"
  ELSE
    OUTPUT "Adult"
ENDIF
```

A nested IF is an IF statement inside another IF's THEN/ELSE branch (shown by extra indentation).

CASE OF ... OTHERWISE ... ENDCASE

```
CASE OF Choice
  1 : Answer  $\leftarrow$  Num1 + Num2
  2 : Answer  $\leftarrow$  Num1 - Num2
  OTHERWISE OUTPUT "Please enter a valid choice"
ENDCASE
```

Three loop (iteration) structures

<i>Loop</i>	<i>Use</i>	<i>Iterations</i>
FOR ... TO ... NEXT	Set number of repetitions (count-controlled)	known in advance

REPEAT ... UNTIL	Post-condition – test at the END	always runs at least once
WHILE ... DO ... ENDWHILE	Pre-condition – test at the START	may run zero times

Exam tip: REPEAT...UNTIL exits when the condition becomes TRUE. WHILE...DO exits when the condition becomes FALSE. Don't mix these up!

7.3–7.4 Purpose of Algorithms & Standard Methods of Solution

You must recognise/write these five standard algorithms in pseudocode:

1. Totalling

```
Total ← 0
FOR Counter ← 1 TO ClassSize
    Total ← Total + StudentMark[Counter]
NEXT Counter
```

2. Counting

```
PassCount ← 0
FOR Counter ← 1 TO ClassSize
    INPUT StudentMark
    IF StudentMark > 50 THEN PassCount ← PassCount + 1
NEXT Counter
```

3. Maximum, minimum, average

Initialise Max very low / Min very high (or set both to the first item in the list), then compare every item.

```
MaximumMark ← StudentMark[1]
MinimumMark ← StudentMark[1]
FOR Counter ← 2 TO ClassSize
    IF StudentMark[Counter] > MaximumMark THEN MaximumMark ← StudentMark[Counter]
    IF StudentMark[Counter] < MinimumMark THEN MinimumMark ← StudentMark[Counter]
NEXT Counter
Average ← Total / ClassSize
```

4. Linear search

Inspects each item in turn until a match is found or the list ends. Uses a Boolean flag variable.

```

Found ← FALSE
Counter ← 1
REPEAT
  IF Name = StudentName[Counter]
    THEN Found ← TRUE
    ELSE Counter ← Counter + 1
  ENDIF
UNTIL Found OR Counter > ClassSize

```

5. Bubble sort

Compares adjacent pairs and swaps if out of order; repeats passes until a pass makes no swaps.

```

First ← 1
Last ← 10
REPEAT
  Swap ← FALSE
  FOR Index ← First TO Last - 1
    IF Temperature[Index] > Temperature[Index+1]
      THEN
        Temp ← Temperature[Index]
        Temperature[Index] ← Temperature[Index+1]
        Temperature[Index+1] ← Temp
        Swap ← TRUE
      ENDIF
  NEXT Index
  Last ← Last - 1
UNTIL (NOT Swap) OR Last = 1

```

Common mistake: forgetting Last decreases each pass (the last element is already sorted correctly) — omitting this doesn't break the sort but makes it less efficient. Forgetting the temp variable when swapping loses data!

7.5 Validation & Verification

Validation: automated check by a program that data is REASONABLE before it's accepted (doesn't guarantee correctness).

Verification: checking data has been copied ACCURATELY from one place to another (e.g. input matches the source).

<i>Validation check</i>	<i>What it checks</i>
Range check	value is between an upper & lower value (e.g. 0–100)
Length check	exact or min/max number of characters
Type check	data is of the correct type (e.g. whole number)
Presence check	a value has actually been entered (not blank)
Format check	characters match a pattern, e.g. CUB9999
Check digit	extra digit calculated from the others – catches mistyped/transposed/missing/extra digits (used in ISBN, barcodes, VIN)

<i>Verification method</i>	<i>How it works</i>
Double entry	data typed twice (maybe by different people); system compares the two
Screen/visual check	user visually checks entered data against the source before confirming

7.6 Test Data

<i>Type</i>	<i>Meaning</i>	<i>Example (range 0–100)</i>
Normal	data the program should accept	50
Abnormal / erroneous	data that should be rejected	–12, "eleven"
Extreme	largest/smallest value still ACCEPTED	0, 100
Boundary	a pair either side of a limit – one accepted, one rejected	–1 (rejected), 0 (accepted)

7.7 Trace Tables & Dry Runs

A dry run = manually working through an algorithm step by step with test data. A trace table records the value of every variable each time it changes, plus an OUTPUT column (only filled when something is actually output).

Exam tip: only write a NEW value in a variable's column when it actually changes on that line – leave the cell blank otherwise. Watch out for questions using single-letter variables (A, B, C, X) – work out the purpose of the algorithm from what it does, not the names!

7.8–7.9 Identifying, Writing & Amending Algorithms

Trace tables reveal logic errors (e.g. wrong comparison operator, loop counter off by one, missing ENDIF). Steps for writing your own algorithm:

1. Clearly specify the problem (purpose + tasks)
2. Decompose into sub-problems: setup → input → processing → storage → output
3. Decide how data will be obtained, stored and displayed
4. Design a structure diagram
5. Choose flowchart or pseudocode
6. Construct it – use meaningful names, precise conditions (e.g. `Counter >= 10`, not "ten or over")
7. Test with Normal, Abnormal and Boundary data using trace tables
8. Correct any errors found and repeat

Extension (A-Level taster) – Stacks & Queues: An ADT (Abstract Data Type) = a collection of data + operations on it.

- Stack – LIFO (Last In First Out). push() adds, pop() removes. Base Pointer never moves; Top Pointer moves.
- Queue – FIFO (First In First Out). enqueue() adds, dequeue() removes. Both Front and End Pointers move.

Key Terms

Abstraction

keeping only key elements, discarding unneeded detail

Decomposition

breaking a problem into smaller parts

Top-down design

breaking a system into sub-systems repeatedly

Algorithm

an ordered set of steps to solve a problem

Flowchart

diagram of steps & order for a task

Pseudocode

algorithm shown in English-like code, no strict syntax

Linear search

checks each item in turn for a match

Bubble sort

repeatedly swaps adjacent out-of-order items

Validation

automated reasonableness check on input

Verification

check that copying was accurate

Normal/Abnormal/Extreme/Boundary data

the 4 types of test data

Trace table

records variable values during a dry run

Practice Questions

Q1. Explain the difference between validation and verification, giving one example of each. [4]

Q2. A program accepts marks between 0 and 50 inclusive. State suitable normal, abnormal, extreme and boundary test data for this program (give two boundary values and expected results). [6]

Q3. Write pseudocode for a linear search that searches an array `Name[1:20]` for a value input by the user, and outputs whether it was found and its position. [5]

Q4. Dry run the following algorithm using the test data 8, 3, 15, 6, showing a full trace table (columns: Counter, Total, X, OUTPUT): [6]

```
Total ← 0
Counter ← 0
REPEAT
    INPUT X
    Total ← Total + X
    Counter ← Counter + 1
UNTIL Counter = 4
OUTPUT Total
```

Q5. State three symbols used in a flowchart and what each one represents. [3]

Q6. Explain, using an example, what is meant by decomposition and why it is a useful technique when designing algorithms. [4]

Q7. The following pseudocode contains errors. Identify three of them. [6]

```
Total ← 0
FOR Counter ← 1 TO 10
    REPEAT
        INPUT Number
        UNTIL Number < 0
        Total ← Total + Number
    NEXT Number
```

Mark Scheme

A1. Validation = automated check that data is reasonable/sensible before being accepted (1); e.g. range check that a mark is between 0–100 (1). Verification = checking data has been copied accurately from one source to another (1); e.g. double entry of a password (1).

A2. Normal: e.g. 25 (1). Abnormal: e.g. -5 or "abc" (1). Extreme: 0 and 50 (1). Boundary: -1 rejected & 0 accepted (1); 50 accepted & 51 rejected (1). 1 mark for correctly labelling each type.

A3. Award marks for: OUTPUT/INPUT the search value (1); Found flag initialised FALSE (1); REPEAT/loop structure with correct exit condition (Found OR Counter>20) (1); correct IF comparison and Counter increment (1); correct final OUTPUT of found/not found and position (1).

A4. Correct row for each of the 4 iterations showing Counter 1,2,3,4 and Total 8,11,26,32 with X values 8,3,15,6 (4 marks, 1 per row); final OUTPUT 32 (1); correct table headers/layout (1).

A5. Any three: Terminator (oval) = start/stop (1); Process (rectangle) = an action/assignment (1); Input/Output (parallelogram) = data entry or display (1); Decision (diamond) = choice with two labelled exits (1).

A6. Decomposition = breaking a complex problem into smaller sub-problems/parts (1) which can be broken down further until each part is simple enough to solve (1). Example: e.g. "getting dressed" broken into select items / remove old clothes / put on in order (1). Benefit: makes a large problem manageable / allows different programmers to work on different parts (1).

A7. Any three: REPEAT...UNTIL condition Number<0 means loop only ends on a negative number, but positive numbers are expected (1); NEXT should refer to Counter, not Number (1); Total is only updated once per outer loop but negative "stop" value shouldn't be added to Total (1); no OUTPUT statement for the total (1).

— end of Chapter 7 —